

Day 15

Linear Search

Professional Notes with Algorithms, Logic, Dry Runs, and Python & JavaScript Solutions



Prepared By
Mahesh Kumar Pothu

May 17, 2026

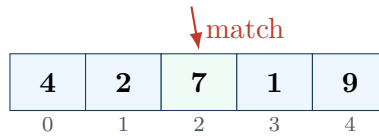
Contents

1	Core Concept Focus	2
2	How to Read, Think, and Build Logic	2
3	Class Questions	3
3.1	1. Find an Element in an Array Using Linear Search	3
3.2	2. Find the First Occurrence of an Element	4
3.3	3. Find the Last Occurrence of an Element	4
3.4	4. Count How Many Times an Element Appears	5
3.5	5. Find All Indexes Where the Element Appears	6
3.6	6. Linear Search in Array of Objects	6
4	Homework Questions	7
4.1	1. Check If an Element Exists in a 2D Array	7
4.2	2. Find the Minimum Value Using Linear Scan	8
4.3	3. Find the Maximum Value Using Linear Scan	9
4.4	4. Find the First Element Greater Than X	9
4.5	5. Check if Array is Strictly Increasing	10
4.6	6. Find the First String That Starts With a Given Character	10
5	Edge Cases and Exam Checklist	11

1 Core Concept Focus

What is Linear Search?

Linear search is the simplest searching technique. It checks every element one by one from left to right until the required value or condition is found. It does not require sorted data.



4	2	7	1	9
0	1	2	3	4

Linear scan checks index 0, then 1, then 2

Universal Linear Search Pattern

1. Start from the first index.
2. Compare the current element with the target value or required condition.
3. If it matches, decide what to return: index, value, object, boolean, count, or position.
4. If it does not match, move to the next element.
5. If the loop ends without success, return a safe failure value such as `-1`, `false`, `null`, or an empty array.

Complexity

Best Case	$O(1)$ when the first element is the answer.
Worst Case	$O(n)$ when the answer is last or absent.
Space	$O(1)$ for simple search; $O(k)$ when storing all matching indexes.
Works On	Unsorted arrays, strings, arrays of objects, 2D arrays, and condition-based checks.

2 How to Read, Think, and Build Logic

Step 1: How to Read the Problem

Read the question slowly and underline four things: **input**, **output**, **condition**, and **return type**. In linear search problems, the condition may be equality, greater than, starts with, matching object property, or a 2D position.

Step 2: How to Understand the Problem

Ask these questions before writing code:

1. What data am I scanning: array, string list, object array, or 2D array?
2. What should I compare at each step?
3. Should I stop at the first match or scan the full array?
4. What should I return if the answer is found?
5. What should I return if the answer is not found?

Step 3: Build Logic First on Paper

Before coding, write a small dry run table with columns like **index**, **current value**, **condition result**, and **action**. This makes the loop logic clear and prevents mistakes.

Paper Logic Template

```

Start from index 0
For each element:
    Check the required condition
    If condition is true:
        Return the required answer
If loop finishes:
    Return failure value
  
```

Step 4: Convert Paper Logic into Code

Every line of paper logic becomes one part of the program:

Start from index 0	Write a loop from 0 to length minus 1.
Check condition	Write an if statement inside the loop.
Return answer	Use <code>return</code> when the required value is found.
Return failure	Write the final <code>return</code> after the loop.

Basic Pattern in Python and JavaScript

```

# Python
for i in range(len(arr)):
    if arr[i] == target:
        return i
return -1

// JavaScript
for (let i = 0; i < arr.length; i++) {
    if (arr[i] === target) return i;
}
return -1;
  
```

3 Class Questions

1. Find an Element in an Array Using Linear Search

Input: [4, 2, 7, 1, 9], element = 7

Output: Found at index 2

Algorithm and Logic

Scan each index. At every step ask: “Is current element equal to 7?” Stop immediately when the answer is yes.

4	2	7	1	9
0	1	2	3	4
not match	not match	match	not checked	not checked

JavaScript

```
function linearSearch(arr, element) {  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] === element) return `Found at index ${i}`;  
  }  
  return "Not found";  
}  
console.log(linearSearch([4, 2, 7, 1, 9], 7));
```

Python

```
def linear_search(arr, element):  
    for i in range(len(arr)):  
        if arr[i] == element:  
            return f"Found at index {i}"  
    return "Not found"  
  
print(linear_search([4, 2, 7, 1, 9], 7))
```

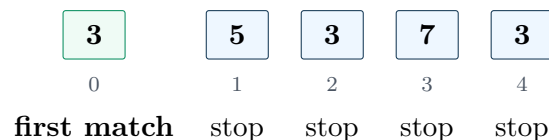
2. Find the First Occurrence of an Element

Input: [3, 5, 3, 7, 3], search = 3

Output: index 0

Algorithm and Logic

The first occurrence means the earliest index. Return as soon as the first match is found; do not continue scanning.



JavaScript

```
function firstOccurrence(arr, search) {  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] === search) return i;  
  }  
  return -1;  
}  
console.log(firstOccurrence([3, 5, 3, 7, 3], 3));
```

Python

```
def first_occurrence(arr, search):  
    for i in range(len(arr)):  
        if arr[i] == search:  
            return i  
    return -1  
  
print(first_occurrence([3, 5, 3, 7, 3], 3))
```

3. Find the Last Occurrence of an Element

Input: [3, 5, 3, 7, 3], search = 3

Output: index 4

Algorithm and Logic

Keep updating `lastIndex` whenever a match appears. After scanning the whole array, `lastIndex` contains the final match.

3	5	3	7	3
0	1	2	3	4
last=0	last=0	last=2	last=2	last=4

JavaScript

```
function lastOccurrence(arr, search) {
  let lastIndex = -1;
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] === search) lastIndex = i;
  }
  return lastIndex;
}
console.log(lastOccurrence([3, 5, 3, 7, 3], 3));
```

Python

```
def last_occurrence(arr, search):
    last_index = -1
    for i in range(len(arr)):
        if arr[i] == search:
            last_index = i
    return last_index

print(last_occurrence([3, 5, 3, 7, 3], 3))
```

4. Count How Many Times an Element Appears

Input: [1, 2, 2, 3, 2, 4], element = 2

Output: 3 times

Algorithm and Logic

Initialize `count = 0`. Increase it by one for every matching element. The final count is the frequency.

1	2	2	3	2	4
0	1	2	3	4	5
0	1	2	2	3	3

JavaScript

```
function countOccurrences(arr, element) {
  let count = 0;
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] === element) count++;
  }
  return count;
}
console.log(countOccurrences([1, 2, 2, 3, 2, 4], 2));
```

Python

```
def count_occurrences(arr, element):
    count = 0
    for value in arr:
        if value == element:
            count += 1
    return count

print(count_occurrences([1, 2, 2, 3, 2, 4], 2))
```

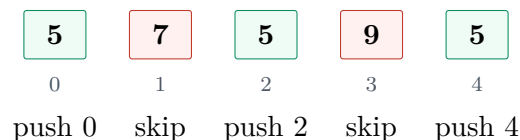
5. Find All Indexes Where the Element Appears

Input: [5, 7, 5, 9, 5], search = 5

Output: [0, 2, 4]

Algorithm and Logic

Create an empty result array. Whenever a match is found, push its index into the result. Do not stop after the first match.



JavaScript

```
function allIndexes(arr, search) {
    const indexes = [];
    for (let i = 0; i < arr.length; i++) {
        if (arr[i] === search) indexes.push(i);
    }
    return indexes;
}
console.log(allIndexes([5, 7, 5, 9, 5], 5));
```

Python

```
def all_indexes(arr, search):
    indexes = []
    for i in range(len(arr)):
        if arr[i] == search:
            indexes.append(i)
    return indexes

print(all_indexes([5, 7, 5, 9, 5], 5))
```

6. Linear Search in Array of Objects

Input: search id = 2

Output: { id: 2, name: "B" }

Algorithm and Logic

For arrays of objects, compare a property instead of the whole object. Here the condition is `students[i].id === 2`.

id:1

0 name A

id:2

1 name B

id:3

2 name C

not match **return object** not checked

JavaScript

```
function findById(items, id) {
  for (let i = 0; i < items.length; i++) {
    if (items[i].id === id) return items[i];
  }
  return null;
}
const data = [
  { id: 1, name: "A" },
  { id: 2, name: "B" },
  { id: 3, name: "C" },
];
console.log(findById(data, 2));
```

Python

```
def find_by_id(items, id_value):
    for item in items:
        if item["id"] == id_value:
            return item
    return None

data = [
    {"id": 1, "name": "A"},
    {"id": 2, "name": "B"},
    {"id": 3, "name": "C"},
]
print(find_by_id(data, 2))
```

4 Homework Questions

1. Check If an Element Exists in a 2D Array

Input: `[[1, 2], [3, 4], [5, 6]]`, `search = 4`**Output:** Found at position (1,1)

Algorithm and Logic

Use two loops. The outer loop selects the row; the inner loop scans columns inside that row. Return row and column as soon as a match is found.

	col 0	col 1
row 0	1	2
row 1	3	4
row 2	5	6

JavaScript

```
function search2D(matrix, search) {
  for (let row = 0; row < matrix.length; row++) {
```



```

    for (let col = 0; col < matrix[row].length; col++) {
        if (matrix[row][col] === search) return `Found at position (${row},${col})`;
    }
    return "Not found";
}
console.log(search2D([[1, 2], [3, 4], [5, 6]], 4));

```

Python

```

def search_2d(matrix, search):
    for row in range(len(matrix)):
        for col in range(len(matrix[row])):
            if matrix[row][col] == search:
                return f"Found at position ({row},{col})"
    return "Not found"

print(search_2d([[1, 2], [3, 4], [5, 6]], 4))

```

2. Find the Minimum Value Using Linear Scan

Input: [9, 3, 5, 1, 7]

Output: 1

Algorithm and Logic

Assume the first element is the minimum. Every time a smaller value appears, update min. At the end, min is the smallest value.

9	3	5	1	7
min=9	min=3	min=3	min=1	min=1

JavaScript

```

function findMin(arr) {
    if (arr.length === 0) return null;
    let min = arr[0];
    for (let i = 1; i < arr.length; i++) {
        if (arr[i] < min) min = arr[i];
    }
    return min;
}
console.log(findMin([9, 3, 5, 1, 7]));

```

Python

```

def find_min(arr):
    if len(arr) == 0:
        return None
    minimum = arr[0]
    for i in range(1, len(arr)):
        if arr[i] < minimum:
            minimum = arr[i]
    return minimum

print(find_min([9, 3, 5, 1, 7]))

```

3. Find the Maximum Value Using Linear Scan

Input: [4, 9, 2, 11, 6]

Output: 11

Algorithm and Logic

Assume the first element is the maximum. Whenever a larger value appears, update max. This is a condition-based linear scan.

4	9	2	11	6
max=4	max=9	max=9	max=11	max=11

JavaScript

```
function findMax(arr) {
  if (arr.length === 0) return null;
  let max = arr[0];
  for (let i = 1; i < arr.length; i++) {
    if (arr[i] > max) max = arr[i];
  }
  return max;
}
console.log(findMax([4, 9, 2, 11, 6]));
```

Python

```
def find_max(arr):
    if len(arr) == 0:
        return None
    maximum = arr[0]
    for i in range(1, len(arr)):
        if arr[i] > maximum:
            maximum = arr[i]
    return maximum

print(find_max([4, 9, 2, 11, 6]))
```

4. Find the First Element Greater Than X

Input: [2, 5, 9, 12, 15], X = 10

Output: 12

Algorithm and Logic

This search is based on a condition, not equality. Check `arr[i] > x`. The first value satisfying the condition is returned immediately.

2	5	9	12	15
$2 \leq 10$	$5 \leq 10$	$9 \leq 10$	$12 > 10$	not checked

JavaScript

```
function firstGreaterThan(arr, x) {
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] > x) return arr[i];
  }
}
```

```
    return null;
}
console.log(firstGreaterThan([2, 5, 9, 12, 15], 10));
```

Python

```
def first_greater_than(arr, x):
    for value in arr:
        if value > x:
            return value
    return None

print(first_greater_than([2, 5, 9, 12, 15], 10))
```

5. Check if Array is Strictly Increasing

Input: [1, 2, 3, 5, 4]

Output: false

Algorithm and Logic

Compare every element with the previous element. For a strictly increasing array, each current value must be greater than the previous value. If `arr[i] <= arr[i - 1]`, return false.

1 < 2	2 < 3	3 < 5	5 < 4 is false
valid	valid	valid	breaks rule

JavaScript

```
function isStrictlyIncreasing(arr) {
    for (let i = 1; i < arr.length; i++) {
        if (arr[i] <= arr[i - 1]) return false;
    }
    return true;
}
console.log(isStrictlyIncreasing([1, 2, 3, 5, 4]));
```

Python

```
def is_strictly_increasing(arr):
    for i in range(1, len(arr)):
        if arr[i] <= arr[i - 1]:
            return False
    return True

print(is_strictly_increasing([1, 2, 3, 5, 4]))
```

6. Find the First String That Starts With a Given Character

Input: ["apple", "ball", "cat", "apply"], char = 'a'

Output: "apple"

Algorithm and Logic

Scan each string and check its first character using `word[0] === char`. Return the first matching string immediately.

apple	ball	cat	apply
starts a	not checked	not checked	not checked

JavaScript

```
function firstStartsWith(words, char) {
  for (let i = 0; i < words.length; i++) {
    if (words[i].length > 0 && words[i][0] === char) return words[i];
  }
  return null;
}
console.log(firstStartsWith(["apple", "ball", "cat", "apply"], "a"));
```

Python

```
def first_starts_with(words, char):
    for word in words:
        if len(word) > 0 and word[0] == char:
            return word
    return None

print(first_starts_with(["apple", "ball", "cat", "apply"], "a"))
```

5 Edge Cases and Exam Checklist

Edge Cases You Must Mention

- **Empty array:** return -1, false, null, or [] depending on the question.
- **Element absent:** complete the full scan and return the failure value.
- **Duplicate elements:** first occurrence stops early; last occurrence scans all elements; count/all indexes scans all elements.
- **Single element array:** compare once and return the result.
- **Array of objects:** compare the required property such as id, name, or marks.
- **2D arrays:** return both row index and column index.

How to Write in an Exam

For every linear search answer, write: **definition** → **algorithm** → **dry run** → **code** → **output** → **complexity**. This structure is clear, complete, and suitable for long-answer questions.



Linear Search = Check One by One Until the Answer is Found